

- 1) Arrays
- 2) Pointers
- 3) Structures

1) ARRAYS

1D

int a[5]

0	1	2	3	4
7	8	9	10	11

a[0] = 7;

a[4] = 11;

Addr	Memory	
0		
4		
8	7	a[0]
12	8	a[1]
16	9	a[2]
20	10	...
24	11	...
28		...

2D

a

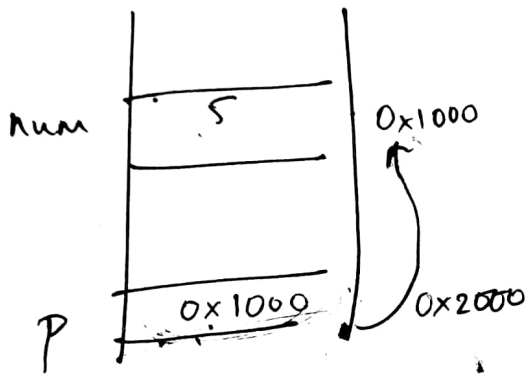
0	1	2	3	→ row 1
4	5	6	7	→ row 2

0
1
2
3
4
5
6
7

row 1 (rows 0-3)
row 2 (rows 4-7)

2) POINTERS

It is a variable that stores address of a variable



int num = 5; address of

int *p = &num

indirection / dereferencing operator

num → 5

p → 0x1000

&num → 0x1000

*p → 5

p = &c;

*p = ~~*c~~ = c

2) Pointers and function arguments

→ By value

```
void swap ( int px, int py ) {
    int temp;
    temp = px;
    px = py;
    py = temp;
}
```

a
[2]

b
[3]

px
[3]

py
[2]

swap(a, b) 2, 3

```
void swap ( int *px, int *py ) {
    int temp;
```

int *p = &a

```
int *temp;
temp = &px;
px = py;
py = temp;
```

```
temp = *px;
*px = *py;
*py = temp;
```

called
a [3]
b [2]

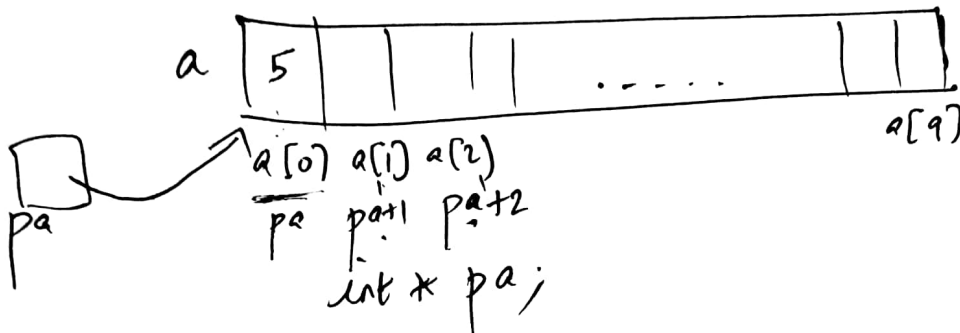
in swap
px []
py []

temp =

swap(&a, &b) swap(2, 3)

(3) Pointers and Arrays

int a[10];



$pa = \underline{\underline{a[0]}}$

OR $pa = \underline{\underline{a}}$

int x = *pa

x = 5

* (pa + i) = pa[i] $\Rightarrow$ pa is a pointer.

* Diff between array names & pointer names

pa = a; ✓

pa++; ✓

a = pa X

a++; X

8t

int len = mystrlen(a); char a[] = "hello"

int strlen(char * s) { S = a

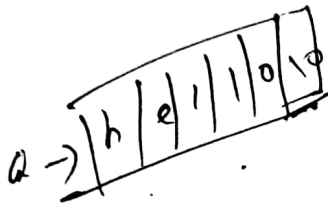
int n;

for (n=0; *s != '\0'; s++) {
n++;

}

return n;

}



NULL pointer

int *p = NULL;

~~#include~~ stdio.h

#define NULL 0

Pointer Arithmetic / operations

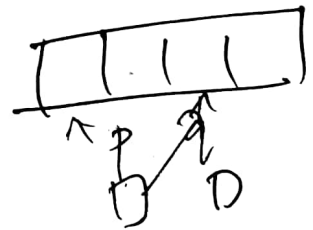
a) Assignment of pointers of same type ✓ P = q

b) +/- a ptr and an integer ✓

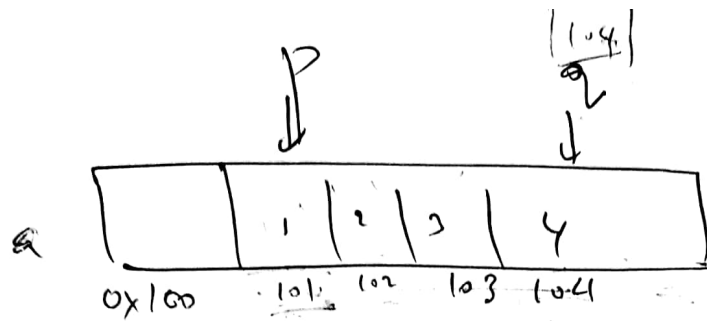
P = P + 1

P = P - 1

c) subtract or compare 2 ptrs of same array.



P < Q



$$n = q - p + 1 = 104 - 101 + 1 = 4$$

~~$p + 3 - p + 1 = 4$~~

$p < q$

d) Assigning or comparing to 0

$p = 0$ ✓

$p = \text{NULL}$ ✓

Illegal pointer operations

p, q are pointers

1. $p + q$ ✗
2. $p * q$ ✗
3. p / q ✗
4. $p + 7.5$ ✗

5. $\text{int } *pn = \&n;$

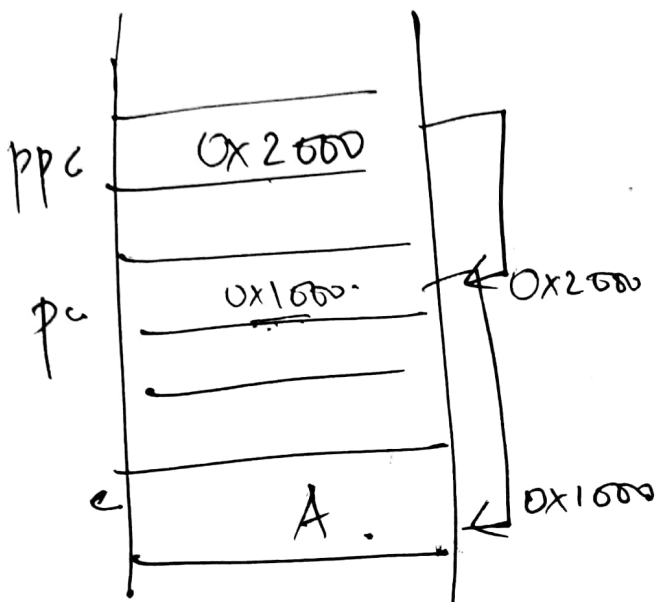
$\text{char } *pc = \&c;$

$pc = pn$ ✗

$pc = (\text{char } *)pn$ ✓

↓
casting $\text{int } *$ to $\text{char } *$

Pointers to pointer



Char c;

Char *pc;

Char **ppc;

c = 'A';

pc = &c;

ppc = &pc;

**ppc = c = *pc = A

ppc = 0x2000

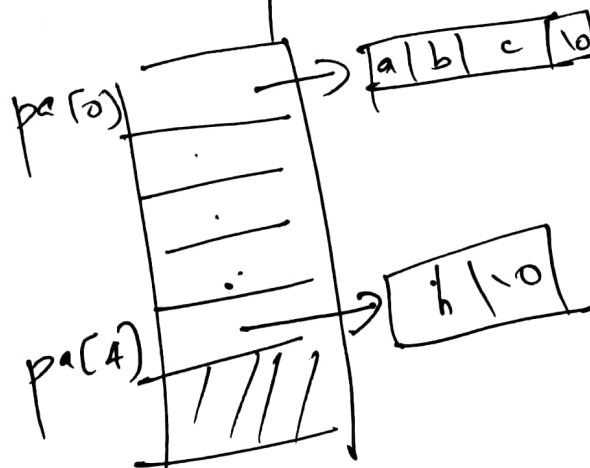
*ppc = 0x1000

**ppc = *(0x1000)
= A

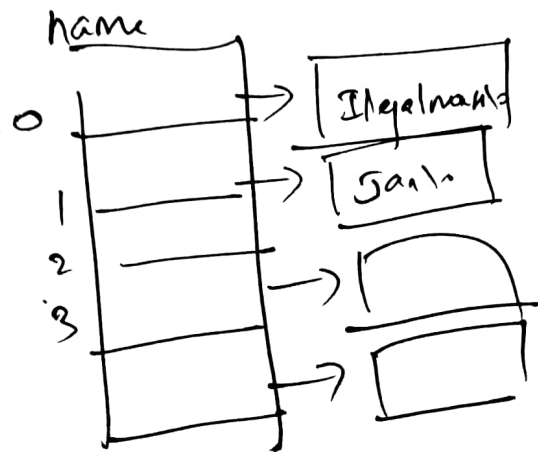
Pointer Arrays

Char *pa[5];

[Char pa[5] = {"", " ", " ", " ", " "}]
[Char *pa]



`char * name[] = { "Illegal month", "Jan", "Feb", "Mar" }`



`char aname[][15] = { "Illegal month", "Jan", "Feb", "Mar" }`

2d char
array



Command Line Arg

```
int main ( int argc, char *argv[] )
```

2

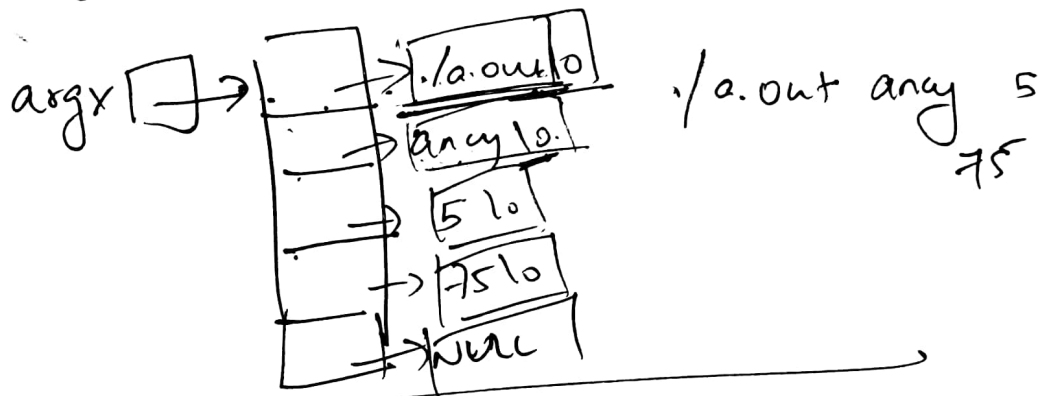
==

~

argc → Argument Count

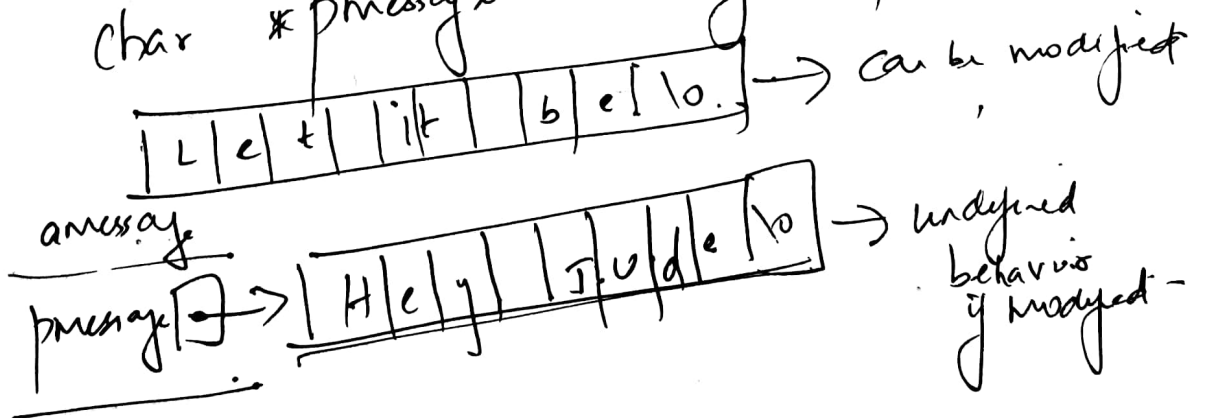
argv → Argument Vector

char arr[] = "any"



char amessage[] = "Let it be"

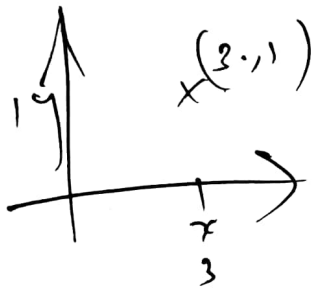
char *pmessage = "Hey Jude";



Division

5 x $\boxed{\text{divide}}$ $\rightarrow$ quotient x/y 2
2 y $\rightarrow$ remainder $x \% y$ 1

Structures



struct point {
 int x;
 int y;
};

$x, y \rightarrow$ members of the structure

point $\rightarrow$ structure tag/name

a struct declaration — defines a type.

struct point p1, p2;

|| $\rightarrow$ int x1, x2;

Accessing members

struct point pt;

$\rightarrow$ structure member operator

$\left. \begin{array}{l} \text{pt.x} \\ \text{pt.y} \end{array} \right\}$ used to access x/y

printf("%d, %d", pt.x, pt.y);

Distance of (x, y) from $(0, 0)$

double dist; sqrt(double);

dist = sqrt(((double)pt.x * pt.x + (double)pt.y * pt.y))

struct point addpoint(struct point p1, struct point p2)

struct point temp;

temp.x = p1.x + p2.x;

temp.y = p1.y + p2.y;

return temp;

}

Pointers to structures

→ Ok to copy small struct like this.

Not OK to copy large structures.

pointers.

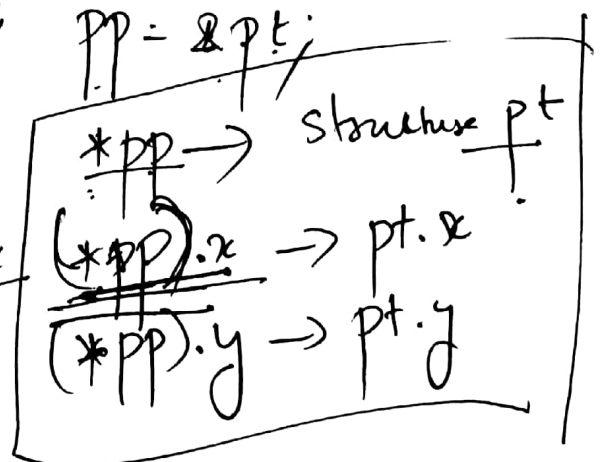
Syntax

struct point *pp;
struct point pt;



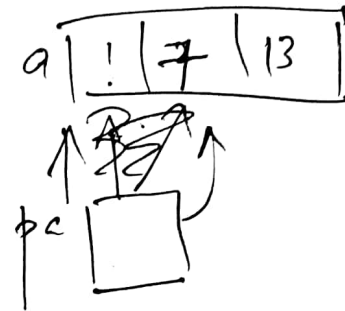
Better syntax

~~pp -> x~~ $\Leftrightarrow$ pt.x
~~pp -> y~~ $\Leftrightarrow$ pt.y



$2 = (*pa)++$
 $1 \quad *pa++$
 $7 \quad *++pa$

$int *pa;$
 $pa = a;$



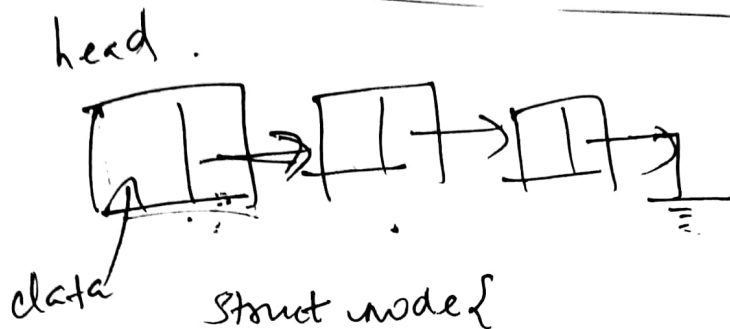
Array of structures

$struct\ studinfo$
 $char *name;$
 $int\ age;$
 $};$

$struct\ studinfo\ ~~stud~~\ cs354_students[70];$

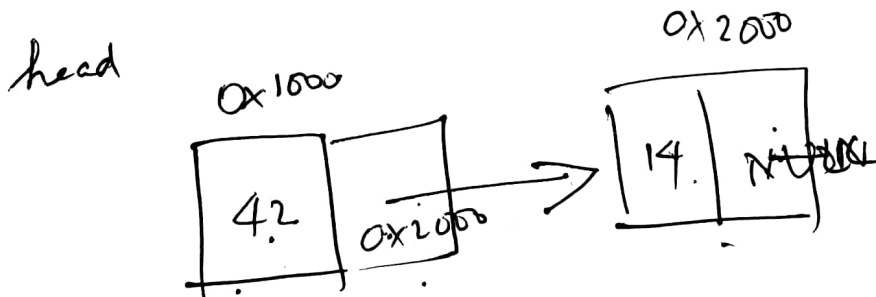
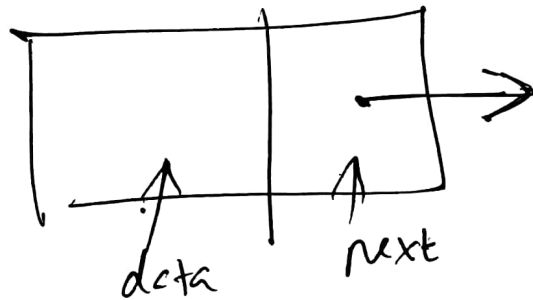
$cs354_students[5].name$
 $\uparrow$
 $i \quad .age$

self referential structures



```
struct node {
    int data;
    struct node *next;
};
```

```
struct node {
    int data;
    struct node next;
};
```



```
struct doubly_linked_node {
    int data;
    struct doubly_linked_node *prev;
    "
    *next;
};
```

